

Departement Informatik
Hochschule Luzern (Schweiz)

Faire Wohnungsvergabe Tool

WEB PROGRAMMING LAB PROJEKT

vorgelegt am Departement Informatik der Hochschule Luzern (Schweiz)

von

Noah Emmenegger

von

Zug (Schweiz)

Web Programming Lab Projekt vorgelegt am Departement Informatik der Hochschule Luzern (Schweiz)

Titel: Faire Wohnungsvergabe Tool

Student: Noah Emmenegger

Studiengang: BSc Informatik

Jahr: 2026

Auftraggeberin / Auftraggeber: Web Programming Lab

Live-Anwendung online ansehen

<https://wohnungsvergabetool.dokploy.codelance.ch>

Kurzfassung

Die Anwendung unterstützt Immobiliengesellschaften und Genossenschaften bei der Verwaltung von Wohnungen, Ausschreibungen und Bewerbungen. Mitarbeitende erfassen Wohnungen, gestalten Bewerbungsformulare mit gewichteten Antwortpräferenzen und veröffentlichen die Formulare über einen Link. Eingereichte Bewerbungen werden in PostgreSQL gespeichert, anhand der festgelegten Präferenzen miteinander verglichen und im Dashboard mit Beiträgen pro Kriterium angezeigt. Ein CSV-Export stellt Bewerbungen und Bewertungen bereit. Die Anwendung ist als Next.js-Monolith mit React, Server Actions und Drizzle ORM umgesetzt und lässt sich mit Docker Compose starten. Automatisierte Unit- und Datenbank-Integrationstests prüfen Gewichtung, Bewertung und Persistenz.

Inhaltsverzeichnis

1	Einführung und Ziele	9
1.1	Aufgabenstellung	9
1.1.1	Fachlicher Umfang	9
1.1.2	Abgrenzung	10
1.2	Qualitätsziele	10
1.3	Stakeholder	10
2	Randbedingungen	12
2.1	Technische Randbedingungen	12
2.2	Organisatorische Randbedingungen	12
2.3	Konventionen	12
3	Kontextabgrenzung	13
3.1	Fachlicher Kontext	13
3.2	Technischer Kontext	13
4	Lösungsstrategie	15
4.1	Technologie-Stack	15
4.2	Architekturansatz	16
4.3	Entwicklungsvorgehen	17
5	Bausteinsicht	18
5.1	Ebene 1: Gesamtsystem	18
5.2	Ebene 2: Next.js-Anwendung	18
5.3	Ebene 2: Persistenz	19
6	Laufzeitsicht	21
6.1	Bewerbung absenden	21
6.2	Bewertete Bewerbungen laden	21
6.3	Startreihenfolge der Datenbankdienste	21
6.4	Aktueller API-Datenfluss	22
7	Verteilungssicht	23
7.1	Container und Laufzeit	23
7.2	Reproduzierbarer Start	24
7.3	Build und Prüfungen	24
8	Querschnittliche Konzepte	25
8.1	Domänenmodell und Persistenz	25
8.2	API- und Schnittstellendesign	25
8.3	Benutzeroberfläche und Responsive Design	25
8.4	Fehlerbehandlung und Logging	25
8.5	Sicherheit	25
8.6	Testkonzept	26
9	Architekturentscheidungen	27
9.1	ADR-001: Next.js statt getrenntem React-Frontend und Express-Backend	27
9.2	ADR-002: PostgreSQL und gemeinsamer Start mit Docker Compose	28
9.3	ADR-003: Drizzle ORM statt Prisma	29
10	Qualitätsanforderungen	30
10.1	Qualitätsbaum	30

10.2	Qualitätsszenarien	30
10.3	Evaluationsergebnisse	31
10.4	Lighthouse-Score	32
11	Risiken und technische Schulden	33
12	Glossar	34
13	Fazit und Reflexion	35
13.1	Was ist gut gelaufen?	35
13.2	Wo lagen die Herausforderungen?	35
13.3	Einsatz von KI	35
13.4	Was würde ich beim nächsten Mal anders machen?	36
13.5	Zielerreichung	36
14	Arbeitsjournal	37
15	Anhang	39
15.1	Abgabe und Softwareartefakte	39

Abkürzungsverzeichnis

Abkürzung	Bedeutung
CMS	Content Management System
HTML	Hypertext Markup Language
CSS	Cascading Style Sheets

Abbildungsverzeichnis

Abbildung 1	Fachlicher Kontext der Wohnungsvergabe-Plattform	13
Abbildung 2	Bausteinsicht des Gesamtsystems	18
Abbildung 3	Lighthouse-Ergebnis der Live-Anwendung	32

Tabellenverzeichnis

Tabelle 1	Schichten und Ausführungsorte der Next.js-Anwendung	19
-----------	---	----

Code-Verzeichnis

Code 1 Laufzeitdienste und Datenfluss der Compose-Umgebung	23
--	----

1 Einführung und Ziele

Dieses Kapitel beschreibt den fachlichen Anlass, den Umfang, die wichtigsten Qualitätsziele und die beteiligten Personen des Projekts.

1.1 Aufgabenstellung

Immobilien-gesellschaften und Genossenschaften möchten Wohnungsvergaben möglichst fair und nachvollziehbar durchführen. Heute müssen Ausschreibungen, Bewerbungsformulare und eingehende Bewerbungen häufig mit mehreren voneinander getrennten Hilfsmitteln verwaltet und manuell verglichen werden.

Im Rahmen des Projekts entstand eine webbasierte Plattform für die Vorbereitung und Durchführung von Wohnungsvergaben. Mitarbeitende einer Gesellschaft erstellen damit Wohnungsausschreibungen und individuelle Bewerbungsformulare. Sie definieren Antwortpräferenzen und deren Gewichtung. Eingehende Bewerbungen werden zentral gespeichert, anhand dieser Regeln miteinander verglichen und sortiert.

Die Plattform unterstützt die Entscheidungsfindung, trifft aber keine endgültige Vergabeentscheidung. Diese bleibt bei der zuständigen Person. Damit die Entscheidung überprüfbar bleibt, zeigt das System für jede Bewerbung transparent auf, wie die Gesamtbewertung zustande gekommen ist.

1.1.1 Fachlicher Umfang

Der ursprüngliche Projektvorschlag enthält die nach MoSCoW priorisierten User Stories. Die implementierte Anwendung deckt folgende Abläufe ab:

- Registrierung und Anmeldung von Mitarbeitenden sowie Verwaltung von Gesellschaften und Wohnungen.
- Erstellen, Bearbeiten, Veröffentlichen und Pausieren von Ausschreibungen mit individuellen Formularfeldern.
- Definition von Antwortpräferenzen und Gewichten für die Bewertung.
- Öffentliches Einreichen von Bewerbungen über einen Ausschreibungslink.
- Anzeige der Bewerbungen mit Rangfolge und Beiträgen der einzelnen Kriterien.
- Export der Bewerbungen und Bewertungen als CSV.

1.1.2 Abgrenzung

Die Systemgrenze umfasst keine automatische Vergabeentscheidung und keine externen Bewertungsdaten:

- Die Plattform trifft keine endgültige Vergabeentscheidung ohne menschliche Bestätigung.
- Es wird keine künstliche Intelligenz zur Bewertung oder Vorhersage der Eignung von Bewerber/innen eingesetzt.
- Es wird kein ausschreibungsübergreifender Score für Bewerber/innen erstellt.
- Es werden keine externen Datenquellen oder sozialen Netzwerke zur Bewertung verwendet.

1.2 Qualitätsziele

Prio	Qualitätsziel	Begründung	Nachweis
1	Nachvollziehbarkeit	Die Bewertung ergibt sich aus Antworten, Kriterien und Gewichtungen.	Unit- und Integrationstests
2	Benutzbarkeit	Die Kernaufgaben sind auf Desktop und Mobile verständlich bedienbar.	Responsive Layouts und verständliche Rückmeldungen
3	Zuverlässigkeit	Ausschreibungen und Bewerbungen werden persistent und konsistent gespeichert.	Datenbank-Integrationstest
4	Änderbarkeit	Die Anwendung ist in klar benannte Module gegliedert.	Bausteinsicht und automatisierte Tests

1.3 Stakeholder

Stakeholder	Erwartung	Bezug zum System
Mitarbeitende einer Immobiliengesellschaft oder Genossenschaft	Einfache Verwaltung von Ausschreibungen und Bewerbungen sowie nachvollziehbare Entscheidungsgrundlagen	Hauptbenutzer/innen des geschützten Verwaltungsbereichs

Stakeholder	Erwartung	Bezug zum System
Bewerber/innen	Ein verständliches, responsives und zuverlässig funktionierendes Bewerbungsformular	Füllen ein öffentlich geteiltes Formular aus
Verantwortliche der Gesellschaft	Ein fairer, dokumentierbarer Vergabeprozess; die endgültige Entscheidung bleibt in ihrer Verantwortung	Verantworten Kriterien und Vergabeentscheid
Entwickler/in und Betreiber/in	Verständliche Struktur, testbare Fachlogik und reproduzierbarer Betrieb	Entwickelt, wartet und betreibt das System
Dozent/in	Nachvollziehbare Erfüllung der Projektanforderungen	Bewertet Artefakt, Vorgehen und Dokumentation

2 Randbedingungen

Dieses Kapitel enthält Vorgaben, die den Lösungsraum des Projekts einschränken. Gewählte Technologien und Architekturansätze werden dagegen in Kapitel 4 beschrieben.

2.1 Technische Randbedingungen

- Die Anwendung muss Daten persistent in einer Datenbank speichern.
- Die Benutzeroberfläche muss auf Desktop, Tablet und Mobile responsiv nutzbar sein.
- Automatisierte Unit-, Integrations- und/oder E2E-Tests müssen passend zu den Projektrisiken eingesetzt werden.
- Ein Produktions-Build muss über eine öffentliche URL oder mit `docker compose up` reproduzierbar startbar sein.
- Der durchschnittliche Lighthouse-Score muss für Mobile und Desktop mindestens 90 betragen.
- Sämtliche Artefakte werden in einem Git-Repository abgegeben.

2.2 Organisatorische Randbedingungen

- Das Projekt wird als Einzelarbeit mit einem vorgesehenen persönlichen Arbeitsaufwand von rund 60 Stunden durchgeführt.
- Die Entwicklung erfolgt strukturiert und systematisch.
- Wesentliche Architekturentscheidungen müssen nachvollziehbar begründet werden.
- Die Dokumentation richtet sich an Software Engineers und bleibt kurz, präzise und diagrammorientiert.

2.3 Konventionen

- Quellcode und Dokumentation werden gemeinsam mit Git versioniert.
- Die Architekturdokumentation orientiert sich an der Kapitelstruktur von arc42.
- Fachbegriffe werden im gesamten System und in der Dokumentation einheitlich verwendet; projektspezifische Begriffe werden im Glossar erläutert.
- Eine User Story gilt erst als abgeschlossen, wenn sie umgesetzt, angemessen getestet und in der Dokumentation beziehungsweise im Arbeitsjournal berücksichtigt ist.

3 Kontextabgrenzung

Dieses Kapitel grenzt die Wohnungsvergabe-Plattform von ihren Benutzergruppen und der technischen Umgebung ab. In der ersten Version bestehen keine Schnittstellen zu externen Fachsystemen oder Datenquellen.

3.1 Fachlicher Kontext

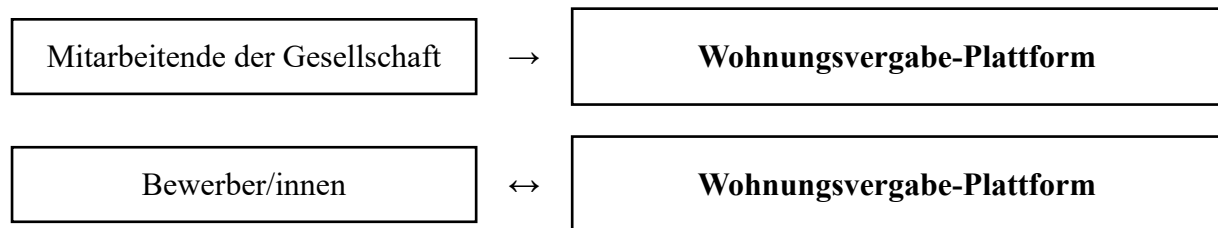


Abbildung 1: Fachlicher Kontext der Wohnungsvergabe-Plattform

Akteur	Schnittstelle	Ausgetauschte Informationen
Mitarbeitende der Gesellschaft	Geschützte Weboberfläche	Wohnungen, Ausschreibungen, Formulare, Kriterien, Gewichtungen, Bewerbungen und Exporte
Bewerber/innen	Öffentlich geteiltes Webformular	Formularfelder und Bewerbungsdaten

Die Gesellschaft legt die Bewertungsregeln fest und erhält die aufbereiteten Bewertungen. Bewerber/innen übermitteln die im jeweiligen Formular verlangten Angaben. Die endgültige Vergabeentscheidung erfolgt ausserhalb des Systems durch die verantwortliche Person.

3.2 Technischer Kontext

Die Benutzer/innen greifen mit einem aktuellen Webbrowser auf die Anwendung zu. Lokal ist sie über HTTP auf Port 3000 erreichbar. Der Next.js App Router ordnet die Anfragen den Seiten und serverseitigen Endpunkten zu. Benutzeroberfläche, serverseitige Validierung und Fachlogik laufen in einer Next.js-Anwendung; es gibt keinen separaten Express-Server. Die Anwendung persistiert die Daten über ein ORM in PostgreSQL. Datenexporte werden als Datei über die Weboberfläche bereitgestellt.

Kommunikations-partner	Kanal/Protokoll	Bemerkung
Browser der Mitarbeitenden	HTTP lokal; HTML, CSS und JavaScript	Zugriff auf den authentisierten Verwaltungsbereich
Browser der Bewerber/innen	HTTP lokal; HTML, CSS und JavaScript	Zugriff über einen öffentlichen Ausschreibungslink
Next.js-Anwendung	Interne Modulaufrufe	Routing, Rendering, serverseitige Endpunkte, Fachlogik und Datenzugriff in einem auslieferbaren Artefakt
PostgreSQL-Datenbank	Datenbankprotokoll über ORM	Persistenz für Ausschreibungen, Formulare, Kriterien, Bewerbungen und Bewertungen
Dateisystem des Browsers	Dateidownload	Export der Bewerbungen und ihrer Bewertungen

Externe Datenquellen, soziale Netzwerke und KI-Dienste liegen ausserhalb der Systemgrenze und werden nicht angebunden.

4 Lösungsstrategie

Dieses Kapitel fasst die grundlegenden Entscheidungen zusammen, mit denen die fachlichen Ziele und Qualitätsziele erreicht werden. Detaillierte und später revidierbare Entscheidungen werden in Kapitel 9 als Architecture Decision Records festgehalten.

4.1 Technologie-Stack

Bereich	Technologie	Begründung und Zielbeitrag
Web-Framework	Next.js mit App Router und TypeScript	Routing, Frontend und Backend können in einer Anwendung entwickelt und als gemeinsames Artefakt ausgeliefert werden. TypeScript unterstützt die konsistente Modellierung der fachlichen Daten.
Benutzeroberfläche	React und Tailwind CSS	Komponentenbasierte Umsetzung der Verwaltungsoberfläche und der dynamischen, responsiven Bewerbungsformulare.
Backend	Next.js Route Handlers und/oder Server Actions auf Node.js	Serverseitige Validierung, Fachlogik und Datenzugriff bleiben innerhalb des Monolithen gebündelt; ein separater Express-Service entfällt.
Datenbank	PostgreSQL	Relationale und transaktionale Speicherung der miteinander verknüpften Ausschreibungen, Kriterien und Bewerbungen.
Datenzugriff	Drizzle ORM	Typsicherer, SQL-naher Datenzugriff; Tabellen und Migrationen bleiben direkt bei den TypeScript-Modellen.
Authentisierung	Benutzeranmeldung für Gesellschaften	Schützt den Verwaltungsbereich und die Bewerbungsdaten.
Tests	Node.js-Test-Runner für Unit- und Datenbank-Integrationstests	Gewichtung, Bewertungsregeln, Ranking, Persistenz und Berechtigungen werden automatisch geprüft.
Betrieb	Docker Compose	Compose startet Web-Anwendung, Migration und PostgreSQL in definierter Reihenfolge.

Bereich	Technologie	Begründung und Zielbeitrag
		ge. Ein Volume persistiert die Datenbankdaten.
Versionsverwaltung	Git und GitHub	Nachvollziehbare Entwicklung und gemeinsame Ablage aller Artefakte.

4.2 Architekturansatz

Die Anwendung ist als modular strukturierter Monolith umgesetzt. Der Next.js App Router, React-Komponenten, serverseitige Endpunkte und Fachlogik befinden sich in derselben Next.js-Anwendung und werden gemeinsam ausgeliefert. Ein eigenständiges Express-Backend entfällt. Innerhalb des Monolithen sind Ausschreibungen, Formulare, Bewertung, Bewerbungen, Benutzerverwaltung und Export in eigenen Modulen gegliedert.

Next.js Server Components sind der Standard für Layouts und nicht interaktive Darstellung. Navigation, Footer und datenlesende Seiten werden serverseitig gerendert. Client Components bilden nur die interaktiven Teilbäume, die Browserzustand, Ereignisbehandlung oder unmittelbare Aktualisierungen benötigen, beispielsweise dynamische Formularfelder, Filter und Sortierungen. Damit wird nur so viel JavaScript wie nötig an den Browser ausgeliefert, ohne auf eine reaktive Bedienung zu verzichten.

Die Bewertung erfolgt regelbasiert. Für jedes gewichtete Feld werden die Antworten der Bewerbungen paarweise verglichen: Eine bessere Antwort erhält die Gewichtung als Punkte, ein Gleichstand die Hälfte. Die Summe der Beiträge ergibt den Gesamtwert. Die Rangfolge hängt damit auch von den übrigen Bewerbungen ab. Das Dashboard und der CSV-Export weisen die Beiträge der einzelnen Kriterien aus.

PostgreSQL speichert die fachlichen Daten. Schreiboperationen werden serverseitig validiert. Docker Compose beschreibt die Laufzeitservices web, migrate und db, ihr internes Netzwerk, die Datenbankkonfiguration und ein persistentes Volume.

Im aktuellen Stand sind Anmeldung, Gesellschaften, Wohnungen, Ausschreibungen, öffentliche Bewerbungsformulare und CSV-Export umgesetzt. Server Components laden geschützte beziehungsweise öffentliche Daten; Server Actions prüfen und speichern Änderungen über Drizzle in PostgreSQL. GET /api/applications bleibt als einfacher technischer Leseendpunkt vorhanden.

4.3 Entwicklungsvorgehen

Die Umsetzung begann mit Datenbankbindung und einem einfachen API-Endpunkt. Anschliessend wurden Anmeldung, Gesellschaften, Wohnungen und der Ablauf von der Ausschreibung bis zur Bewerbung integriert. Antwortpräferenzen, Gewichtung, Rangliste und Export ergänzten diesen Kernablauf. Architekturentscheidungen sind als ADR festgehalten; das Arbeitsjournal dokumentiert die zeitliche Entwicklung.

5 Bausteinsicht

5.1 Ebene 1: Gesamtsystem

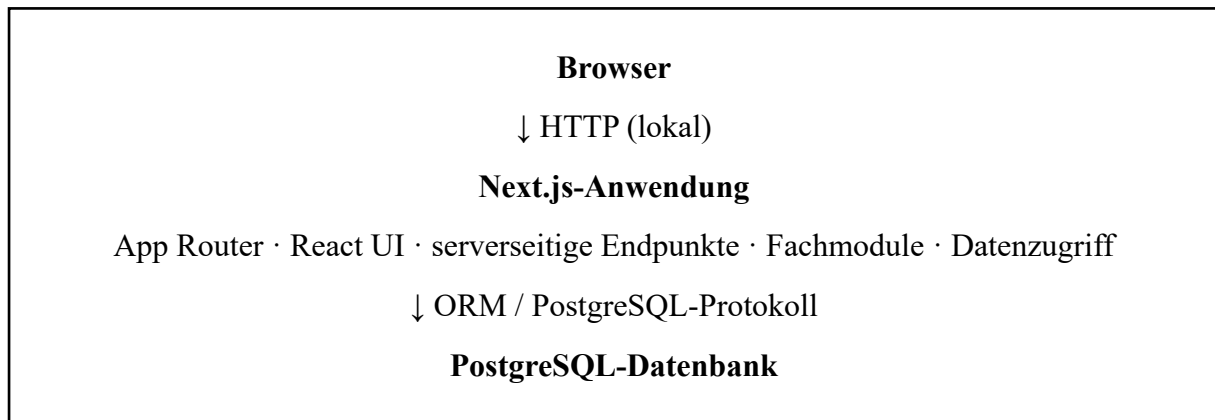


Abbildung 2: Bausteinsicht des Gesamtsystems

Baustein	Verantwortung	Schnittstellen
App Router und UI	Ordnet URLs zu, rendert Seiten und verarbeitet Benutzerinteraktionen.	HTTP im lokalen Betrieb sowie interne Aufrufe
Serverseitige Endpunkte	Authentisierung, Validierung und Orchestrierung der Anwendungsfälle mit Route Handlers und/oder Server Actions.	HTTP oder interne Server-Aufrufe
Fachmodule	Regeln für Ausschreibungen, Formulare, Kriterien, Bewerbungen, Bewertungen und Export.	Typsichere Modulgrenzen
Datenzugriff	Kapselt Abfragen, Transaktionen und Schema-Migrationen.	ORM
PostgreSQL	Persistente, relationale und transaktionale Datenhaltung.	PostgreSQL-Protokoll

5.2 Ebene 2: Next.js-Anwendung

Die Anwendung ist in Schichten mit gerichteten Abhängigkeiten gegliedert. Der App Router und die Komponenten bilden die Darstellung. Server Actions und Route Handlers nehmen Eingaben entgegen und koordinieren Anwendungsfälle. Datenbankfreie Funktionen in `lib/` enthalten Typen, Gewichtungs- und Bewertungsregeln. Die Abfragen in `db/queries/` kapseln

Drizzle und PostgreSQL. Nur serverseitige Einstiegspunkte importieren den Datenzugriff; die Oberfläche erhält Daten als Props oder als Rückgabe einer Server Action.

Ausführungsort	Schicht und Datenfluss
Browser	components/ mit "use client": Formulare, Dialoge, Dashboard-Zustand und Ereignisse
Server	app/ mit Server Components: Routing, initiale Daten und Zugriffsschutz
Server	app/actions/ und app/api/: Autorisierung, Eingabeprüfung und Anwendungsfälle
Beide Seiten	lib/: reine Typen, Gewichtung- und Bewertungsfunktionen ohne Datenbankzugriff
Nur Server	db/queries/ → db/index.ts → PostgreSQL: Abfragen und Persistenz

Tabelle 1: Schichten und Ausführungsorte der Next.js-Anwendung

app/dashboard/layout.tsx prüft die Sitzung und lädt Gesellschaften, Wohnungen und Ausschreibungen auf dem Server. Die Daten gehen an DashboardShell und DashboardProvider, die als Client Components Auswahl, Dialoge und lokalen Zustand verwalten. Die öffentliche Route app/bewerben/[listingId]/page.tsx lädt eine veröffentlichte Ausschreibung serverseitig; ApplicationForm verarbeitet die Eingabe im Browser. Das Formular ruft submitListingApplication als Server Action auf, wo die Eingaben erneut geprüft und gespeichert werden. Der CSV-Export ist ein separater Route Handler unter app/api/listings/[listingId]/applications/export/route.ts.

Die Funktionen in lib/answer-preferences.ts und lib/field-weights.ts sind bewusst unabhängig von React und PostgreSQL. Sie werden sowohl für die Vorschau und Rangliste im Dashboard als auch bei serverseitiger Validierung und beim Export genutzt. Clientseitige Berechnungen dienen der unmittelbaren Anzeige; Berechtigungen und verbindliche Schreiboperationen bleiben auf dem Server. Damit ist die Schichtung eine Trennung der Verantwortlichkeiten innerhalb eines Monolithen, keine vollständig isolierte Domain-Schicht.

5.3 Ebene 2: Persistenz

Das ORM bildet das relationale PostgreSQL-Schema auf TypeScript-Typen ab und verwaltet versionierte Migrationen. Nur serverseitiger Code erhält Zugriff auf die Datenbankverbin-

dung. Bewerbungsantworten und die zum Einreichungszeitpunkt verwendeten Formularfelder werden in `listing_applications` gespeichert. Die Rangfolge ergibt sich aus den Antworten und den aktuellen Bewertungsregeln der Ausschreibung.

`db/schema.ts` ist die verbindliche Drizzle-Definition für Benutzer, Sitzungen, Gesellschaften, Wohnungen, Ausschreibungen und Bewerbungen. SQL-Migrationen liegen versioniert in `drizzle/`. Der Pool und der Drizzle-Client befinden sich in `db/index.ts` und werden nicht in Client Components importiert.

6 Laufzeitsicht

6.1 Bewerbung absenden

1. Eine Bewerberin oder ein Bewerber öffnet die durch den Next.js App Router bereitgestellte Formularroute. Navigation, Footer und Formularrahmen werden serverseitig gerendert.
2. Interaktive Formulareile werden als Client Components aktiviert. Sie prüfen unmittelbar erkennbare Eingabefehler und senden die Bewerbung an einen serverseitigen Endpunkt der Next.js-Anwendung.
3. Die Server Action prüft den Veröffentlichungsstatus der Ausschreibung und validiert die Eingaben erneut.
4. Die Server Action speichert die Antworten und eine Kopie der verwendeten Formularfelder über Drizzle in PostgreSQL. Die Rangfolge wird beim Anzeigen der Bewerbungen aus den gespeicherten Daten berechnet.
5. Bei Erfolg zeigt die Client Component eine Bestätigung. Bei einem erwarteten Fehler bleiben die Eingaben erhalten und die Oberfläche zeigt eine verständliche Meldung.

6.2 Bewertete Bewerbungen laden

1. Eine angemeldete Mitarbeiterin oder ein angemeldeter Mitarbeiter ruft im Verwaltungsbereich eine Ausschreibung auf.
2. Das Dashboard-Layout prüft die Sitzung auf dem Server und lädt die eigenen Gesellschaften, Wohnungen und Ausschreibungen.
3. Für eine ausgewählte Ausschreibung ruft die Client Component eine Server Action auf. Diese prüft die Berechtigung und lädt die Bewerbungen aus PostgreSQL.
4. Die datenbankfreie Bewertungsfunktion berechnet im Browser die Rangfolge für die Anzeige. Der CSV-Export berechnet dieselbe Rangfolge separat auf dem Server.

6.3 Startreihenfolge der Datenbankdienste

1. Docker Compose startet die Services db, migrate und web.
2. PostgreSQL initialisiert das Datenverzeichnis und meldet seinen Zustand über einen Healthcheck.

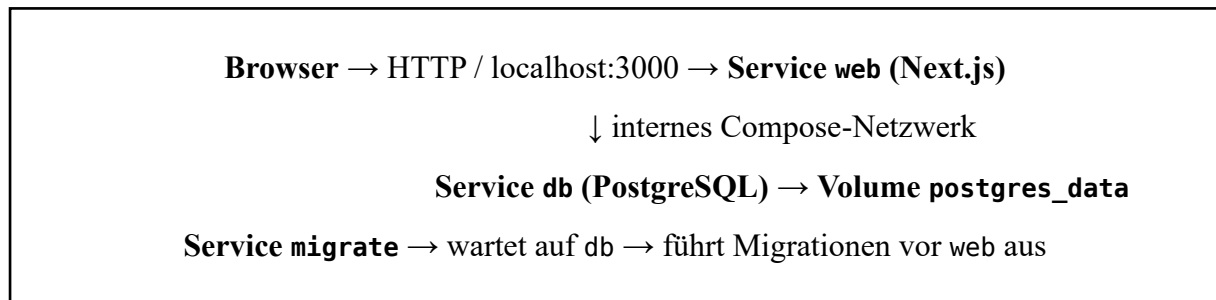
3. Nach erfolgreichem Healthcheck führt migrate die versionierten SQL-Migrationen aus.
Erst nach dessen erfolgreichem Abschluss startet web. Schlägt ein Schritt fehl, startet der Web-Service nicht.

6.4 Aktueller API-Datenfluss

1. Ein Client ruft GET /api/applications auf.
2. Der Route Handler verwendet den serverseitigen Drizzle-Client und führt eine sortierte SELECT-Abfrage auf applications aus.
3. Bei Erfolg antwortet die Route mit { applications: [...] } und Cache-Control: no-store.
4. Bei einem unerwarteten Datenbankfehler wird der technische Fehler serverseitig protokolliert; der Client erhält eine allgemeine HTTP-500-Antwort ohne Verbindungsdetails.

7 Verteilungssicht

7.1 Container und Laufzeit



Code 1: Laufzeitdienste und Datenfluss der Compose-Umgebung

Knoten	Artefakt	Verantwortung
Browser	HTML und Client-JavaScript	Zeigt serverseitig erzeugte Seiten und führt interaktive Client Components aus.
Service web	Next.js-Standalone-Bundle aus dem Dockerfile	Server Components, Server Actions, Route Handlers, Authentisierung und Datenzugriff; intern auf Port 3000.
Service migrate	Eigenes Docker-Build-Target	Wendet mit <code>npm run db:migrate</code> versionierte Drizzle-Migrationen an und beendet sich danach.
Service db	PostgreSQL 17	Speichert relationale Daten; Healthcheck meldet die Bereitschaft. Der Port ist nur lokal an <code>127.0.0.1</code> gebunden.
Docker-Volume	postgres_data	Erhält Daten bei einer Neuerstellung der Container.

Das produktive Docker-Image wird mehrstufig gebaut. Nur das Standalone-Bundle, statische Dateien und öffentliche Assets gelangen in den laufenden Web-Container. Compose startet migrate nach erfolgreichem Datenbank-Healthcheck und web erst nach erfolgreicher Migration. So ist die Migrationsreihenfolge in der aktuellen Compose-Datei technisch festgelegt; fehlgeschlagene Migrationen verhindern den Start von web.

7.2 Reproduzierbarer Start

Voraussetzung ist Docker Compose. Eine `.env`-Datei beziehungsweise sicher gesetzte Umgebungsvariablen liefern Datenbankname, -benutzer, -passwort und gegebenenfalls erlaubte Origins für Server Actions. Echte Zugangsdaten werden nicht in Git eingecheckt. Für den lokalen Browserzugriff ergänzt `docker-compose.local.yml` die Portfreigabe des Web-Services.

```
docker compose -f docker-compose.yml -f docker-compose.local.yml up --build
```

Die Anwendung ist lokal über Port 3000 erreichbar. `docker compose -f docker-compose.yml -f docker-compose.local.yml down` stoppt die Dienste und erhält das Datenbank-Volume. Schemaänderungen werden vorab mit `npm run db:generate` als SQL-Migration erzeugt und versioniert; der einmalige Service `migrate` wendet sie beim nächsten Compose-Start an.

7.3 Build und Prüfungen

Das Dockerfile erstellt ein Next.js-Standalone-Bundle. Die Skripte `npm run typecheck`, `npm run lint`, `npm test` und `npm run build` prüfen Typen, Stilregeln, Bewertungslogik und Produktions-Build. Der Datenbank-Integrationstest wird mit `npm run test:integration` gegen eine separate Testdatenbank ausgeführt.

8 Querschnittliche Konzepte

8.1 Domänenmodell und Persistenz

Drizzle definiert in `db/schema.ts` Benutzer, Sitzungen, Gesellschaften, Wohnungen, Ausschreibungen und Bewerbungen. `listing_applications` speichert Antworten zusammen mit den zum Einreichungszeitpunkt verwendeten Formularfeldern. SQL-Migrationen werden versioniert und beim Compose-Start vor dem Web-Service ausgeführt.

8.2 API- und Schnittstellendesign

`GET /api/hello` und `GET /api/applications` liefern JSON. Der geschützte Route Handler für den Bewerbungs-Export liefert CSV. Fachliche Schreiboperationen laufen über Server Actions. Die Bewerbungsroute ist bewusst nicht zwischengespeichert und gibt bei unerwarteten Fehlern eine allgemeine HTTP-500-Antwort aus; technische Details bleiben im Server-Log.

8.3 Benutzeroberfläche und Responsive Design

Die Anwendung verwendet das Next.js-App-Layout mit deutscher Dokumentensprache und Metadaten. Dashboard, Dialoge und öffentliches Bewerbungsformular sind umgesetzt. Interaktive Bereiche sind Client Components; geschützte Daten werden serverseitig geladen und geprüft.

8.4 Fehlerbehandlung und Logging

Konfigurationsfehler wie eine fehlende `DATABASE_URL` verhindern den Start des Datenbank-Clients. Datenbankfehler der API werden serverseitig mit Kontext geloggt, während der Client keine Zugangsdaten oder SQL-Details erhält.

8.5 Sicherheit

Zugangsdaten liegen in `.env` und sind von Git ausgeschlossen. Docker Compose verlangt ein explizites PostgreSQL-Passwort; der Datenbankport ist nur an `127.0.0.1` gebunden. Die Sitzung wird über ein HTTP-only-Cookie verwaltet. Das Dashboard prüft die Anmeldung, und serverseitige Aktionen und Exporte prüfen die Berechtigung für die jeweilige Ausschreibung. Eingaben werden serverseitig erneut validiert.

8.6 Testkonzept

Teststufe	Zweck	Beispiele/Werkzeuge
Statisch	Typen und Stilregeln prüfen	npm run typecheck und npm run lint
Build	Produktionsbundle und Route-Typen prüfen	npm run build
Unit	Gewichtung, Eingabeprüfung und Ranking prüfen	npm test: automatisierte Tests mit dem Node.js-Test-Runner
Integration	Persistenz und Auswertung mit PostgreSQL prüfen	npm run test:integration mit separater TEST_DATABASE_URL; Ausführung benötigt eine migrierte Testdatenbank

9 Architekturentscheidungen

Entscheidungen werden als kurze Architecture Decision Records (ADR) festgehalten. Eine Entscheidung beschreibt Kontext und Folgen, nicht nur das gewählte Werkzeug.

9.1 ADR-001: Next.js statt getrenntem React-Frontend und Express-Backend

Status	Akzeptiert am 2. September 2026
Kontext	Ursprünglich waren ein React-Frontend und ein separates Express-Backend vorgesehen. Für die Einzelarbeit sollen Routing, Typen, Betrieb und Deployment mit wenig Doppelkonfiguration beherrschbar bleiben. Serverseitige Validierung und geschützter Datenzugriff sind erforderlich.
Betrachtete Optionen	1. React-SPA mit Express-API; 2. Next.js als Full-Stack-Framework; 3. getrennte Frontend- und Backend-Anwendungen in einem Monorepo.
Entscheidung	Next.js mit App Router und TypeScript wird als modularer Monolith eingesetzt. Server Components rendern Navigation, Footer, Layouts und datenlesende Seiten. Client Components übernehmen interaktive Teilbereiche. Route Handlers und/oder Server Actions bilden serverseitige Endpunkte; ein Express-Service entfällt.
Begründung	Next.js bündelt Routing, Rendering und Backend-Funktionen. Gemeinsame Typen und ein Artefakt reduzieren Schnittstellen- und Betriebsaufwand. Serverorientiertes Rendering begrenzt das Browser-JavaScript und passt zum Projektumfang.
Konsequenzen	Weniger Konfiguration und kein separates API-Deployment stehen einer Bindung an Next.js-Konventionen gegenüber. Klare Modulgrenzen, kleine Client-Inseln, ausschliesslich serverseitiger Datenbankzugriff und automatisierte Tests verhindern eine Vermischung der Verantwortlichkeiten.

9.2 ADR-002: PostgreSQL und gemeinsamer Start mit Docker Compose

Status	Akzeptiert am 2. September 2026
Kontext	Ausschreibungen, dynamische Formulare, Kriterien, Gewichtungen, Bewerbungen und Bewertungen stehen in engen Beziehungen und müssen konsistent gespeichert werden. Die lokale Laufzeitumgebung soll auf verschiedenen Rechnern mit einem Befehl reproduzierbar sein.
Betrachtete Optionen	1. Eingebettete Datenbank; 2. lokal manuell installierte PostgreSQL-Datenbank; 3. PostgreSQL als Docker-Service zusammen mit der containerisierten Next.js-Anwendung.
Entscheidung	PostgreSQL wird als Service db betrieben. Die Next.js-Anwendung läuft als Service app. Docker Compose definiert Build, Netzwerk, Healthcheck, Umgebungsvariablen und ein persistentes Datenbank-Volume. Der gesamte Stack startet mit <code>docker compose up --build</code> .
Begründung	PostgreSQL unterstützt relationale Integrität und Transaktionen für das verknüpfte Fachmodell. Docker Compose vereinheitlicht Versionen und Startablauf, ohne eine lokal installierte Datenbank vorauszusetzen.
Konsequenzen	Docker ist für die lokale Ausführung erforderlich und benötigt zusätzliche Ressourcen. Datenbankmigrationen, Secrets sowie Backup und Restore bleiben explizite Betriebsaufgaben. Der Datenbankport wird standardmässig nicht nach aussen veröffentlicht; ein Volume verhindert Datenverlust bei einer Neuerstellung des Containers.

9.3 ADR-003: Drizzle ORM statt Prisma

Status	Akzeptiert am 18. September 2026
Kontext	Die Next.js-Anwendung benötigt einen typsicheren Zugriff auf PostgreSQL sowie versionierte Migrationen. Der technische Kern soll klein bleiben und SQL-Abfragen nachvollziehbar machen.
Betrachtete Optionen	1. Direkter Zugriff mit pg; 2. Prisma; 3. Drizzle ORM.
Entscheidung	Drizzle ORM wird mit dem node-postgres-Treiber eingesetzt. Tabellen werden in <code>db/schema.ts</code> definiert; Drizzle Kit erzeugt und wendet Migrationen im Verzeichnis <code>drizzle/</code> an.
Begründung	Drizzle verbindet aus dem TypeScript-Schema abgeleitete Typen mit SQL-nahen Abfragen. Gegenüber Prisma benötigt es keinen zusätzlichen Generierungsschritt für einen Client und hält die Abfragen für das kleine Projekt direkt lesbar. Gegenüber reinem pg reduziert es wiederholte Typdefinitionen und strukturiert Schema-Migrationen.
Konsequenzen	Datenbankzugriff bleibt auf Node.js-Routen beschränkt. Schemaänderungen benötigen weiterhin eine erzeugte und kontrolliert angewendete Migration. Teammitglieder müssen die Drizzle-Abfrage- und Migrationskonventionen kennen.

10 Qualitätsanforderungen

10.1 Qualitätsbaum

- Qualität
 - Nachvollziehbarkeit: deterministische Bewertung, erklärbare Teilbewertungen
 - Benutzbarkeit: responsive Bedienung, verständliche Rückmeldungen
 - Änderbarkeit: klare Verantwortlichkeiten, automatisierte Tests
 - Zuverlässigkeit: Datenintegrität, kontrollierte Fehlerbehandlung
 - Effizienz: Lighthouse-Durchschnitt mindestens 90 auf Mobile und Desktop

10.2 Qualitätsszenarien

ID	Merkmal	Szenario	Messgrösse	Nachweis
QS-01	Performance	Eine typische Seite wird in der Produktionsversion aufgerufen.	Lighthouse-Durchschnitt ≥ 90 , Mobile und Desktop	Lighthouse-Bericht
QS-02	Benutzbarkeit	Eine Kernfunktion wird auf einem kleinen Bildschirm ausgeführt.	Ohne horizontales Scrollen; alle Aktionen erreichbar	E2E/Manuell
QS-03	Zuverlässigkeit	Ungültige Daten werden übermittelt.	Keine inkonsistenten Daten; verständliche Fehlermeldung	Integration/E2E
QS-04	Änderbarkeit	Ein neues Bewertungskriterium oder ein neuer Formulartyp wird ergänzt.	Änderung bleibt auf klar abgegrenzte Bausteine beschränkt	Review/Tests
QS-05	Nachvollziehbarkeit	Eine gespeicherte Bewertung wird automatisch bewertet.	Gesamtwert ist aus Antworten, Kriterien und Gewichtungen reproduzierbar; alle Teilwerte werden ausgewiesen	Unit-/E2E-Test

10.3 Evaluationsergebnisse

Am 25.09.2026 bestanden vier automatisierte Unit-Tests für Gewichtung, Validierung und Ranking (`npm test`). Ein PostgreSQL-Integrationstest für persistierte Ausschreibungen, Bewerbungen und ihre Auswertung bestand ebenfalls (`npm run test:integration`). Der Integrationstest lief nach Anwendung der Migrationen gegen eine separate lokale Testdatenbank. Die Testfälle liegen in `tests/`.

10.4 Lighthouse-Score

Der Screenshot zeigt 99 Punkte für Leistung, 95 für Barrierefreiheit, 82 für Best Practices und 100 für SEO. Eine separate Desktop-Messung von /dashboard ergab nahezu gleiche Werte (100/95/81/100).

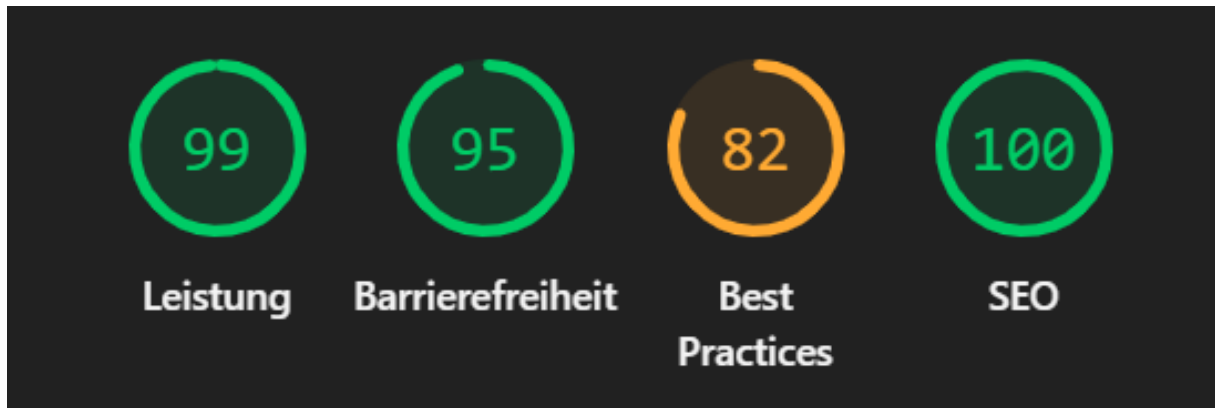


Abbildung 3: Lighthouse-Ergebnis der Live-Anwendung

11 Risiken und technische Schulden

Die folgenden technischen Risiken wurden im lokalen Betrieb berücksichtigt:

Risiko	Umgesetzte Behandlung
Datenbank ist beim Start des Web-Services nicht bereit	PostgreSQL-Healthcheck und Abhängigkeit des Web-Services vom erfolgreichen Migrationslauf
Fehlerhafte Migration beeinträchtigt den Start	Der Service migrate führt versionierte Migrationen aus; web startet erst nach dessen erfolgreichem Abschluss.
Datenbankzugriff aus Browser-Code	Datenbankabfragen liegen in serverseitigen Modulen; Berechtigungen werden bei Server Actions und Exporten geprüft.

12 Glossar

Begriff	Bedeutung im Projekt
App Router	Dateisystembasiertes Routing von Next.js für Seiten, Layouts und serverseitige Endpunkte
Client Component	React-Komponente, die für Browserzustand oder Interaktionen clientseitig ausgeführt wird
CRUD	Erstellen, Lesen, Ändern und Löschen einer Ressource
Docker Compose	Beschreibung und gemeinsamer Start der Container app und db samt Netzwerk, Konfiguration und Volume
Modularer Monolith	Eine gemeinsam ausgelieferte Anwendung, deren fachliche Verantwortlichkeiten intern in klar abgegrenzte Module gegliedert sind
MoSCoW	Priorisierung in Must, Should, Could und Won't
Route Handler	Serverseitiger HTTP-Endpunkt innerhalb des Next.js App Routers
Server Component	Standardmässig serverseitig gerenderte React-Komponente, zum Beispiel für Navigation, Footer oder datenlesende Seiten
Server Action	Serverseitig ausgeführte Next.js-Funktion, die von der Benutzeroberfläche aus aufgerufen werden kann

13 Fazit und Reflexion

13.1 Was ist gut gelaufen?

Ich konnte auf meine Erfahrung aus mehreren Next.js-Anwendungen und mit PostgreSQL zurückgreifen. Dadurch gelang der durchgängige Ablauf von der Erfassung einer Wohnung über die Ausschreibung bis zur Bewerbung und Auswertung zügig. Die Trennung zwischen Oberfläche, Server Actions, Bewertungsfunktionen und Datenbankabfragen macht die Verantwortlichkeiten im Code nachvollziehbar.

Besonders bereichernd fand ich, mich im Unterricht wieder mit den Grundlagen und Eigenheiten von JavaScript auseinanderzusetzen. Nach der Arbeit mit vielen Frameworks fühlte sich dieser Blick zurück auf die Basics für mich fast nostalgisch an und machte zugleich Spass.

13.2 Wo lagen die Herausforderungen?

Am schwierigsten war für mich zunächst die Wahl einer passenden Architektur. Während des Projekts bewarb ich mich auf eine Stelle, bei der Next.js, PostgreSQL und Docker eingesetzt werden. Ich entschied mich bewusst für diesen Stack: Er passte zum Projekt und gab mir Gelegenheit, mein Wissen für das anstehende technische Interview aufzufrischen und zu vertiefen.

Interessant war für mich dabei besonders Drizzle ORM. In früheren Projekten hatte ich Datenbankzugriffe stärker getrennt und teils direkt mit SQL gearbeitet. Mit Drizzle konnte ich nun Schema, typsichere Abfragen und versionierte Migrationen in einem für mich neuen Ansatz verbinden. Die dynamischen Formularfelder, Antwortpräferenzen und Gewichte verlangten ausserdem eine eindeutige serverseitige Validierung. Auch Migrationen und die Startreihenfolge der Docker-Container mussten aufeinander abgestimmt werden.

13.3 Einsatz von KI

Ich habe KI während der Entwicklung als Unterstützung eingesetzt. Die Verantwortung für das Ergebnis blieb aber bei mir: Vorschläge und erzeugter Code mussten geprüft und an meine Architektur angepasst werden. Mehrfach schlug die KI etwas anderes vor, als ich beabsichtigt hatte. Gerade deshalb war es wichtig, ihre Antworten kritisch zu hinterfragen und technische Entscheidungen selbst zu treffen.

13.4 Was würde ich beim nächsten Mal anders machen?

Rückblickend hätte ich konkrete Akzeptanzkriterien für die priorisierten User Stories früher festgelegt. Auch die Testfälle für Gewichtung und Ranking hätte ich parallel zur ersten Implementierung der Bewertungslogik schreiben können. So wären Abweichungen zwischen Projektvorschlag, Anwendung und Dokumentation früher sichtbar geworden.

13.5 Zielerreichung

Umgesetzt wurden die Verwaltung von Gesellschaften und Wohnungen, individuelle Ausschreibungen, öffentliche Bewerbungsformulare, die gewichtete Auswertung eingegangener Bewerbungen und ein CSV-Export. Die Anwendung speichert ihre Daten in PostgreSQL und kann mit Docker Compose gestartet werden. Vier Unit-Tests und ein Datenbank-Integrationstest bestanden am 25.09.2026; Typecheck, Lint und Produktions-Build liefen ebenfalls erfolgreich. Die Bewertungsbeiträge sind für jede Bewerbung im Dashboard und im Export nachvollziehbar.

14 Arbeitsjournal

Datum	Stunden	Tätigkeiten/Ergebnis	Kumuliert
31.08.2026	3.0	Grundstruktur der arc42-Dokumentation und erstes Typst-Template angelegt.	3.0
01.09.2026	5.0	Dokumentationsvorlage integriert und angepasst; Kapitelstruktur, Titelseite und ersten Projektvorschlag ausgearbeitet. Architekten miteinander verglichen und evaluiert	8.0
02.09.2026	7.0	Projektvorschlag und Architekturkapitel zu Kontext, Lösung, Bausteinen und Qualität überarbeitet; PDFs erstellt.	15.0
16.09.2026	4.0	Umsetzung vorbereitet und Projektstruktur, Datenhaltung und lokale Entwicklungsumgebung geplant.	19.0
17.09.2026	6.0	Next.js-Projekt aufgesetzt; Docker-Compose mit PostgreSQL, Drizzle ORM konfiguriert und integriert und Datenbankbindung eingerichtet.	25.0
18.09.2026	4.0	Beispiel-API mit Drizzle-Abfragen und erster Datenbankmigration umgesetzt; Ablauf im README beschrieben.	29.0
19.09.2026	4.0	API- und Docker-Konfiguration bereinigt; Architekturentscheidungen und technische Konzepte dokumentiert.	33.0
22.09.2026	5.0	Datenmodell und Abläufe für Benutzer, Firmen und Wohnungen vorbereitet; Dashboard-Struktur ausgearbeitet.	38.0
23.09.2026	9.0	Registrierung, Anmeldung, Firmenverwaltung und Dashboard umgesetzt; Wohnungsverwaltung, Ausschreibungsformular und Bewerbungsablauf ergänzt.	47.0

Datum	Stunden	Tätigkeiten/Ergebnis	Kumuliert
24.09.2026	8.0	Antwortpräferenzen und Feldgewichtung ergänzt; dynamischen Regler und Filter für die beste Übereinstimmung umgesetzt.	55.0
25.09.2026	8.0	Datenbankzugriff und Docker-Deployment korrigiert; Migration repariert sowie Bewerbungs-Export und Import einer Beispielwohnung ergänzt. Doku ergänzt und abgeschlossen	63.0
Total	63.0	Persönlicher Arbeitsaufwand	63.0

15 Anhang

15.1 Abgabe und Softwareartefakte

Artefakt	Fundort
Git-Repository	github.com/NoahEmmenegger/weblab-projekt
Startanleitung	README.md, Abschnitt „Docker Compose“
Automatisierte Tests	tests/assessment.test.ts und tests/ database.integration.test.ts; Ausführung im README.md be- schrieben
Projektvorschlag	Doc/projektvorschlag.typ